

Neural Networks: Regularization (L1 & L2)

This notebook continues from the Neural Networks coding questions and introduces **regularization** — a technique to prevent overfitting by penalizing large weights during training.

```
import numpy as np
```

Background: Why Regularization?

When training a neural network, the model can overfit — learning the training data so well that it performs poorly on unseen data. Regularization addresses this by adding a **penalty term** to the loss function that discourages overly large weight values.

The two most common forms are:

- **L2 (Ridge)**: penalizes the sum of squared weights → encourages small, spread-out weights
- **L1 (Lasso)**: penalizes the sum of absolute weights → encourages sparse weights (many become exactly 0)

The regularized loss becomes:

$$L_{reg} = L_{original} + \lambda \cdot \Omega(W)$$

where λ (lambda) is the regularization strength hyperparameter.

Question 10: L2 Regularization Penalty

Implement a function that computes the **L2 regularization penalty** (also called weight decay):

$$\Omega_{L2}(W) = \frac{1}{2} \sum_i w_i^2$$

The function should accept weights **W** and regularization strength **lambda_** and return the scalar penalty value.

Hint: The ½ factor is included so the gradient simplifies cleanly to W.

Answer 10

```
def l2_penalty(W, lambda_):  
    """Compute L2 regularization penalty: 0.5 * lambda * sum(W^2)"""  
    return 0.5 * lambda_ * np.sum(W ** 2)
```

```
# Test
W = np.array([0.5, -1.2, 0.8])
lambda_ = 0.1
print(f"L2 Penalty: {l2_penalty(W, lambda_):.4f}")

L2 Penalty: 0.1165
```

Question 11: L1 Regularization Penalty

Implement a function that computes the **L1 regularization penalty**:

$$\Omega_{L1}(W) = \sum_i |w_i|$$

The function should accept weights `W` and regularization strength `lambda_` and return the scalar penalty value.

Answer 11

```
def l1_penalty(W, lambda_):
    """Compute L1 regularization penalty: lambda * sum(|W|)"""
    return lambda_ * np.sum(np.abs(W))

# Test
W = np.array([0.5, -1.2, 0.8])
lambda_ = 0.1
print(f"L1 Penalty: {l1_penalty(W, lambda_):.4f}")

L1 Penalty: 0.2500
```

Question 12: Regularized MSE Loss

Using your `mse` function from Question 1 and the penalty functions above, implement a **regularized loss** function that combines MSE with either L1 or L2 regularization.

The function signature should be:

```
def regularized_loss(y_hat, y, W, lambda_, mode='l2')
```

where `mode` is either `'l1'` or `'l2'`.

Answer 12

```
def mse(y_hat, y):
    """Mean squared error (from Q1)"""
    return np.mean((y_hat - y) ** 2)
```

```

def regularized_loss(y_hat, y, W, lambda_, mode='l2'):
    """
    Compute MSE loss with L1 or L2 regularization.

    Parameters:
        y_hat : predictions
        y      : true labels
        W      : weight vector
        lambda_ : regularization strength
        mode   : 'l1' or 'l2'
    """
    base_loss = mse(y_hat, y)
    if mode == 'l2':
        penalty = l2_penalty(W, lambda_)
    elif mode == 'l1':
        penalty = l1_penalty(W, lambda_)
    else:
        raise ValueError("mode must be 'l1' or 'l2'")
    return base_loss + penalty

# Test
y_true = np.array([1.0, 2.0, 3.0])
y_pred = np.array([1.1, 2.2, 2.9])
W = np.array([0.5, -1.2, 0.8])
lambda_ = 0.1

print(f"MSE only: {mse(y_pred, y_true):.4f}")
print(f"MSE + L2 penalty: {regularized_loss(y_pred, y_true, W, lambda_, mode='l2'):.4f}")
print(f"MSE + L1 penalty: {regularized_loss(y_pred, y_true, W, lambda_, mode='l1'):.4f}")

MSE only:          0.0200
MSE + L2 penalty:  0.1365
MSE + L1 penalty:  0.2700

```

Question 13: L2 Gradient (Weight Decay Update)

During backpropagation, regularization adds an extra gradient term. For L2 regularization, the gradient of the penalty with respect to weights is:

$$\frac{\partial \Omega_{L2}}{\partial W} = \lambda \cdot W$$

Implement a function `l2_gradient(W, lambda_)` that returns this gradient. Then implement `update_weights_l2` that applies a gradient descent step with L2 regularization:

$$W \leftarrow W - \alpha \cdot (\nabla L + \lambda W)$$

where α is the learning rate and ∇L is the gradient from the original loss.

Answer 13

```
def l2_gradient(W, lambda_):
    """Gradient of L2 penalty with respect to W"""
    return lambda_ * W

def update_weights_l2(W, grad_loss, learning_rate, lambda_):
    """
    Gradient descent step with L2 regularization.

    W <- W - lr * (grad_loss + lambda * W)
    """
    return W - learning_rate * (grad_loss + l2_gradient(W, lambda_))

# Test
W = np.array([0.5, -1.2, 0.8])
grad_loss = np.array([0.1, -0.3, 0.2]) # gradient from MSE
lr = 0.1
lambda_ = 0.1

W_updated = update_weights_l2(W, grad_loss, lr, lambda_)
print(f"Original weights: {W}")
print(f"Updated weights: {W_updated}")

Original weights: [ 0.5 -1.2  0.8]
Updated weights:  [ 0.485 -1.158  0.772]
```

Question 14: L1 Gradient (Sign Update)

For L1 regularization, the gradient of the penalty is the **sign** of the weights:

$$\frac{\partial \Omega_{L1}}{\partial W} = \lambda \cdot \text{sign}(W)$$

Implement `l1_gradient(W, lambda_)` and `update_weights_l1` following the same pattern as Question 13.

Note: Unlike L2, the L1 gradient does not shrink weights proportionally — it subtracts a constant amount, pushing small weights all the way to zero.

Answer 14

```
def l1_gradient(W, lambda_):
    """Gradient of L1 penalty with respect to W"""
    return lambda_ * np.sign(W)
```

```
def update_weights_l1(W, grad_loss, learning_rate, lambda_):
    """
    Gradient descent step with L1 regularization.

     $W \leftarrow W - lr * (grad\_loss + lambda * sign(W))$ 
    """
    return W - learning_rate * (grad_loss + l1_gradient(W, lambda_))

# Test – compare L1 vs L2 on the same weights
W = np.array([0.5, -1.2, 0.8])
grad_loss = np.array([0.1, -0.3, 0.2])
lr = 0.1
lambda_ = 0.1

W_l1 = update_weights_l1(W, grad_loss, lr, lambda_)
W_l2 = update_weights_l2(W, grad_loss, lr, lambda_)

print(f"Original weights: {W}")
print(f"L1 updated: {W_l1}")
print(f"L2 updated: {W_l2}")

Original weights: [ 0.5 -1.2  0.8]
L1 updated:      [ 0.48 -1.16  0.77]
L2 updated:      [ 0.485 -1.158  0.772]
```

Question 15: Comparing L1 vs L2 Sparsity

One key difference between L1 and L2 regularization is that **L1 produces sparse solutions** — it tends to drive some weights exactly to zero, effectively performing feature selection.

Write a function `simulate_regularization(W_init, grad_loss, learning_rate, lambda_, epochs)` that applies weight updates for a fixed number of epochs using both L1 and L2, then:

1. Returns the final weight vectors for both methods
2. Counts how many weights are effectively zero (absolute value $< 1e-4$) in each case

Use the same `grad_loss` at every step to simulate a simplified training scenario.

Answer 15

```
def simulate_regularization(W_init, grad_loss, learning_rate, lambda_,
    epochs):
    """
    Simulate training with L1 and L2 regularization.

    Returns:
```

```

    W_l1 (np.ndarray): final weights under L1
    W_l2 (np.ndarray): final weights under L2
    zeros_l1 (int):    number of near-zero weights under L1
    zeros_l2 (int):    number of near-zero weights under L2
"""
W_l1 = W_init.copy()
W_l2 = W_init.copy()

for _ in range(epochs):
    W_l1 = update_weights_l1(W_l1, grad_loss, learning_rate,
lambda_)
    W_l2 = update_weights_l2(W_l2, grad_loss, learning_rate,
lambda_)

    zeros_l1 = np.sum(np.abs(W_l1) < 1e-4)
    zeros_l2 = np.sum(np.abs(W_l2) < 1e-4)

    return W_l1, W_l2, zeros_l1, zeros_l2

# Test with weights of varying magnitude
W_init = np.array([2.0, 0.05, -1.5, 0.02, 0.8])
grad_loss = np.zeros(5) # no task loss – only regularization drives
updates
lr = 0.05
lambda_ = 0.1
epochs = 100

W_l1, W_l2, zeros_l1, zeros_l2 = simulate_regularization(W_init,
grad_loss, lr, lambda_, epochs)

print(f"Initial weights:    {W_init}")
print(f"L1 final weights:    {np.round(W_l1, 4)}")
print(f"L2 final weights:    {np.round(W_l2, 4)}")
print(f"\nZero weights – L1: {zeros_l1}, L2: {zeros_l2}")

Initial weights:    [ 2.    0.05 -1.5   0.02  0.8 ]
L1 final weights:    [ 1.5 -0.   -1.   -0.   0.3]
L2 final weights:    [ 1.2115  0.0303 -0.9087  0.0121  0.4846]

Zero weights – L1: 2, L2: 0
(L1 is expected to produce more zeros – sparse solution)

```